Scope-Based Authorization Demo — KongAir Dev Portal

Scope-Based Authorization with Dynamic Client Registration

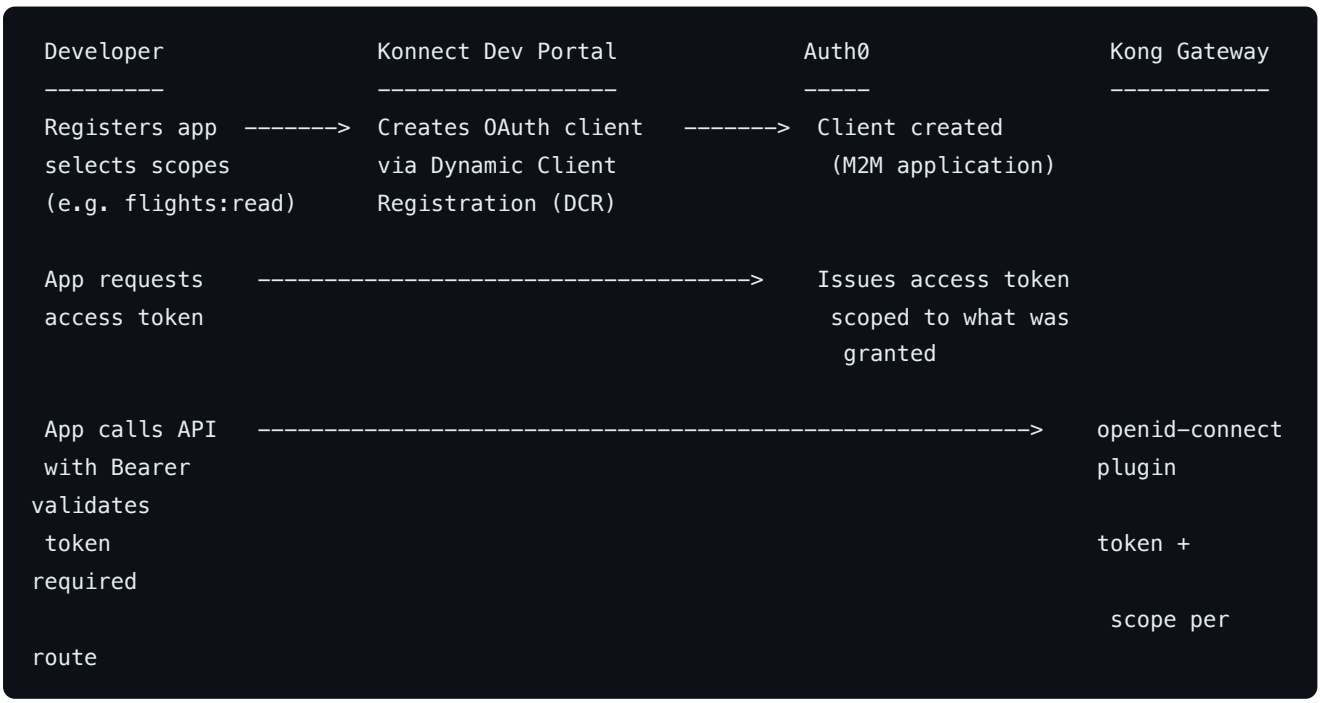
KongAir Developer Portal — Kong Konnect

Overview

This document describes the setup and verification of scope-based authorization for the KongAir Developer Portal's Bookings and Flights APIs, using OAuth 2.0 Dynamic Client Registration (DCR) against Auth0.

The goal: instead of every registered application receiving the same static API key, each developer now authenticates via OAuth 2.0 client credentials and selects which scopes their application needs at registration time (for example, read-only access to Flights, or read/write access to Bookings). The Kong Gateway enforces those scopes independently per route, so a token issued for one API cannot be used against another, and a read-only token cannot perform write operations.

Architecture



Two authorization layers work together:

- **Konnect Portal Teams** (pre-existing) — control who can *see and register* for an API in the Developer Portal.
- **OAuth scopes** (this work) — control what an *issued token* is allowed to do when it reaches the Kong Gateway, independent of the Portal Teams model.

Setup Summary

1. Auth0 Dynamic Client Registration provider

A Konnect `DCR provider` was configured against the customer's Auth0 tenant. It uses a dedicated Auth0 Machine-to-Machine application (management credentials only — never seen by end developers) to create a new OAuth client in Auth0 automatically whenever a developer registers an application in the Portal.

Developer-managed scopes is enabled: each application registration lets the developer choose which scopes to request, rather than every application receiving a fixed, identical set.

2. Application Auth Strategy

A single OIDC application auth strategy is used for both APIs, offering four scopes:

Scope	Grants
`bookings:read`	Read access to the Bookings API
`bookings:write`	Write access to the Bookings API
`flights:read`	Read access to the Flights API
`flights:write`	Write access to the Flights API

Tokens are issued for audience `https://kongair.com/api/v1`, matching the API resource defined in Auth0.

3. Gateway enforcement

Each route on the Bookings and Flights services carries an `openid-connect` plugin requiring the matching scope:

Route	Required scope
`GET /bookings`	`bookings:read`
`POST /bookings`	`bookings:write`
`GET /flights`	`flights:read`
`GET /flights/{flightNumber}`	`flights:read`
`GET /flights/{flightNumber}/details`	`flights:read`
`GET /bookings/health` , `GET /flights/health`	*(unauthenticated — health checks)*

This enforcement is generated automatically from each API's OpenAPI specification: any route whose method is a read (`GET` / `HEAD`) requires the `:read` scope, any mutating method (`POST` / `PUT` / `PATCH` / `DELETE`) requires `:write` , and any operation explicitly marked as public in the spec (such as a health check) is left open.

4. Auth0 configuration required

Two settings must exist on the customer's Auth0 tenant for this flow to work end to end:

1. The Auth0 Management API application used for DCR must be granted the `read:resource_servers` permission (in addition to the standard client and client-grant management permissions), which is required for the developer-managed-scopes flow.
2. The `https://kongair.com/api/v1` API resource in Auth0 must define `bookings:read` , `bookings:write` , `flights:read` , and `flights:write` as permissions, with RBAC and "Add Permissions in the Access Token" enabled — otherwise Auth0 has nothing to hand out when a developer selects a scope.

Verification

The following tests were run against the live Kong Gateway data plane. Tokens were obtained from Auth0 via the OAuth 2.0 client-credentials grant, using a test application's Auth0-issued client ID and secret; the example token below is truncated and its signature is not reproduced.

Test token (client-credentials grant, audience `https://kongair.com/api/v1`):


```
{
  "iss": "https://<tenant>.auth0.com/",
  "sub": "<client-id>@clients",
  "aud": "https://kongair.com/api/v1",
  "scope": "flights:read flights:write",
  "gty": "client-credentials"
}
```

#	Request	Expected	Result
1	`GET /flights/health` — no token	Open, no auth required	**200 OK**
2	`GET /flights` — no token	Rejected, missing credential	**401 Unauthorized** (`error="invalid_token"`)
3	`GET /flights` — token with `flights:read`	Allowed	**200 OK**
4	`POST /bookings` — same token (no `bookings:*` scope)	Rejected, wrong scope	**403 Forbidden** (`error="insufficient_scope"`)

Test 2 and test 4 responses both carry a `WWW-Authenticate` header identifying the exact failure reason:

```
WWW-Authenticate: Bearer realm="<tenant>.auth0.com", error="invalid_token"
WWW-Authenticate: Bearer realm="<tenant>.auth0.com", error="insufficient_scope"
```

All four results matched expectations: unauthenticated health checks remain open, a missing token is rejected, a correctly-scoped token is accepted, and a token lacking the required scope is rejected specifically for *insufficient scope* rather than a generic authentication failure — confirming the gateway is evaluating scopes per route rather than only checking that a token is present.

Summary

The Developer Portal now issues OAuth 2.0 credentials per application via Auth0 Dynamic Client Registration, with each application scoped to only the APIs and operations its developer selected at registration time. The Kong Gateway independently enforces those scopes at the route level, so access control does not rely on the Developer Portal alone — a valid token that lacks the right scope is rejected at the gateway itself.